

SoC-Based Image Compression System Using JPEG

Artur Miranda Oliveira

1 Introduction

Computational systems make extensive use of graphical systems, especially nowadays with the increasing number of applications involving pattern recognition in images. As is well known, information stored and transmitted by hardware must meet certain specifications, usually related to processing time and limited storage. The same applies to images; moreover, images tend to be especially large. For example, an image with a resolution of 512x512, where each pixel can contain approximately 16 million colors (24 bits), has about 786,432 bytes. This is an extremely high value, especially if the images are to be transmitted and processed in real-time by IoT devices, whose environments often have limited internet connection speed, processing capability, and available energy.

Fortunately, in the real world, images can be compressed by eliminating redundant information or using lossy methods. An example of a lossy method is JPEG, which uses the DCT (Discrete Cosine Transform), and through the coefficients obtained for each cosine component that describes the image, selects those that have the greatest impact on the image construction (only the largest coefficients are kept).

2 Image Representation in Digital Systems

Before delving into the details of image compression, it is important to introduce how a digital system, such as a computer or a *System on Chip* (SoC), interprets and processes an image.

An image is nothing more than a two-dimensional matrix of pixels. The term pixel is an abbreviation for *picture element* [Sal10]. For many engineers, programmers, and users, pixels can be thought of as small squares, and this is generally true for devices such as monitors. However, for other devices, pixels may be circular, rectangular, etc. By definition, a pixel should be considered as a point without dimension. It can be discrete in the case of digital computing systems, or it can be a real value, as in the case of sensors—for example, the analog output of the optical sensor of a camera.

Furthermore, an image can be classified into several types depending on its characteristics. Some of them are:

- **Bi-level (or monochromatic) image:** This is an image where the pixels can have one of two values, usually referred to as black and white. Each pixel in such an image is represented by one bit, making it the simplest type of image.
- **Grayscale image:** A pixel in this type of image can have a value ranging from 0 to $n - 1$, representing one of 2^n shades of gray (or shades of some other color). The value of n is usually compatible with the size of a byte; that is, it is 4, 8, 12, 16, 24, or some other convenient multiple of 4 or 8.
- **Continuous-tone image:** This type of image may contain many similar colors (or shades of gray). When adjacent pixels differ by only one unit, it is

difficult or even impossible for the eye to distinguish their colors. As a result, such an image may contain areas where colors appear to vary continuously as the eye moves across the area. A pixel in this kind of image is represented by a large number (in the case of many grayscale tones) or by three components (in the case of a color image). Continuous-tone images are usually natural images, obtained by photographing with a digital camera or scanning a photograph or painting.

- **Discrete-tone image:** Typically, this is an artificial image. It may have few or many colors but lacks the noise and blur of a natural image. Examples include an artificial object or machine, a page of text, a graph, an animation, or the contents of a computer screen. (Not every artificial image is discrete-tone. A computer-generated image intended to look natural is a continuous-tone image, even though it was artificially created.) Artificial objects, text, and line drawings have sharp and well-defined edges and are therefore highly contrasting with the rest of the image (the background). Adjacent pixels in a discrete-tone image are often identical or vary significantly in value. This type of image does not compress well using lossy methods, as the loss of just a few pixels can render a letter unreadable or change a familiar pattern into an unrecognizable one. Compression methods for continuous-tone images often do not handle sharp edges well, so special methods are needed for efficient compression of such images. Note that a discrete-tone image may be highly redundant, as the same character or pattern may appear many times within the image.

For illustration purposes, below is the representation of a monochromatic image and its pixel matrix in Figure 1, and the representation of a continuous-tone image and its pixel matrix in Figures 2 and 3.

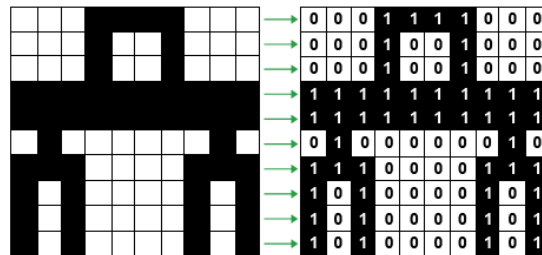


Figure 1: Monochromatic image



Figure 2: Continuous-tone image

52	55	61	66	70	61	64	73
63	59	55	90	109	85	69	72
62	59	68	113	144	104	66	73
63	58	71	122	154	106	70	69
67	61	68	104	126	88	68	70
79	65	60	70	77	68	58	75
85	71	64	59	55	61	65	83
87	79	69	68	65	76	78	94

Figure 3: Pixel matrix of a continuous-tone image

3 Image Compression

An image, after all, exists to be seen by people, so when it is compressed, it is acceptable to lose characteristics of the image to which the human eye is not sensitive. This is one of the main ideas behind various lossy image compression methods.

In general, information can be compressed if it is redundant. However, with lossy compression, we have a new concept, which is compression by removing irrelevance. An image can be compressed with loss by removing irrelevant information (e.g., small color variations that are imperceptible to the human eye), even if the original image has no redundancy.

This work considers three compression methods that are the foundation of JPEG: *Run Length Encoding*, *Discrete Cosine Transform* (DCT), and *Huffman Encoding*.

3.1 DCT

Image transforms are designed to have two main properties: (1) reduce image redundancy by decreasing the size of most pixels, and (2) identify the least important parts of the image by isolating its various frequencies. Intuitively, we associate a frequency with a wave. Water waves, sound waves, and electromagnetic waves have frequencies—but so do pixels in an image.

To understand this concept, it is necessary to introduce the DCT. The one-dimensional DCT is defined as:

$$G_f = \sqrt{\frac{2}{n}} C_f \sum_{t=0}^{n-1} p_t \cos \left[\frac{(2t+1)f\pi}{2n} \right]$$

where,

$$C_f = \begin{cases} \frac{1}{\sqrt{2}}, & f = 0, \\ 1, & f > 0, \end{cases} \quad \text{for } f = 0, 1, \dots, n-1. \quad (1)$$

The input is a set of n data values p_t (pixels, audio samples, or other data), and

the output is a set of n DCT transformation coefficients (or weights) G_f .

However, a more intuitive way to view the DCT is to interpret it in matrix form. A transform is defined as a mapping from a function, vector, or number to another value in a different domain or basis:

$$\mathcal{T}\{f(t)\} = F(\omega)$$

such that,

$$f(t) = \mathcal{T}^{-1}\{F(\omega)\}$$

In matrix form:

$$\mathbf{X} = \mathbf{T}\mathbf{x}$$

where $\mathbf{x}$ is the input vector (function or original data), $\mathbf{T}$ is the transformation matrix, and $\mathbf{X}$ is the output vector (transformed coefficients). If the matrix $\mathbf{T}$ is composed of cosines, the vector $\mathbf{x}$ can be seen as a weighted sum of cosines, with the weights given by $\mathbf{X}$. Consider the example where $n = 3$:

$$\begin{bmatrix} G_0 \\ G_1 \\ G_2 \end{bmatrix} = \begin{bmatrix} \cos 0 & \cos 0 & \cos 0 \\ \cos \frac{\pi}{6} & \cos \frac{3\pi}{6} & \cos \frac{5\pi}{6} \\ \cos \frac{2\pi}{6} & \cos \frac{4\pi}{6} & \cos \frac{6\pi}{6} \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ p_2 \end{bmatrix} = \mathbf{D} \cdot \mathbf{p}. \quad (2)$$

If the angles selected for the DCT are $\theta_j = (j + 0.5) \frac{\pi}{n}$, then the components of each vector $\mathbf{v}_k$ are $\cos \left[k \left(j + 0.5 \right) \frac{\pi}{n} \right]$. Each vector $\mathbf{v}_k$ is a row of $\mathbf{D}$. The equation above can be written with the rows of matrix $\mathbf{D}$ normalized by $\sqrt{3}, \sqrt{1.5}, \sqrt{1.5}$, respectively. Thus, $\mathbf{p}$ can be written as follows:

$$\begin{bmatrix} 0.5773 & 0.7071 & 0.4083 \\ 0.5773 & -0.0000 & -0.8165 \\ 0.5773 & -0.7071 & 0.4083 \end{bmatrix} \cdot \begin{bmatrix} G_0 \\ G_1 \\ G_2 \end{bmatrix} = \mathbf{p}$$

In other words, $\mathbf{p}$ is a weighted sum of discrete sinusoids with frequencies 0, $\frac{\pi}{6}$, and $\frac{2\pi}{6}$.

To further illustrate, consider three additional examples. The first example is $\mathbf{p} = (p, p, p)$. The three components of $\mathbf{p}$ are identical, so they correspond to a zero frequency. The product $\mathbf{D} \cdot \mathbf{p}$ yields the frequency coefficients $(1.7322p, 0, 0)$, indicating the absence of high frequencies. The second example is $\mathbf{p} = (p, 0, -p)$. The components of $\mathbf{p}$ slowly vary from p to $-p$, so this vector contains a low frequency. The product $\mathbf{D} \cdot \mathbf{p}$ gives the coefficients $(0, 1.4142p, 0)$, confirming this result. The third example is $\mathbf{p} = (p, -p, p)$. The components of $\mathbf{p}$ vary from p to $-p$, and then back to p , meaning this vector contains a high frequency. The product $\mathbf{D} \cdot \mathbf{p}$ produces $(0, 0, 1.6329p)$, again indicating the correct frequency.

For an 8-element input $\mathbf{p}$, a pixel-based graphical representation of the DCT can be seen in Figure 4, where the elements of $\mathbf{p}$ can be viewed as a weighted sum of each vector $\mathbf{v}_k$, that is, $\mathbf{p} = \sum_i w_i \mathbf{v}_i$. The DCT, given an input vector, returns the weights that each $\mathbf{v}_k$ must have so that the weighted sum of these vectors reproduces the input sequence. The vectors $\mathbf{v}_k$ can be interpreted as basis images.

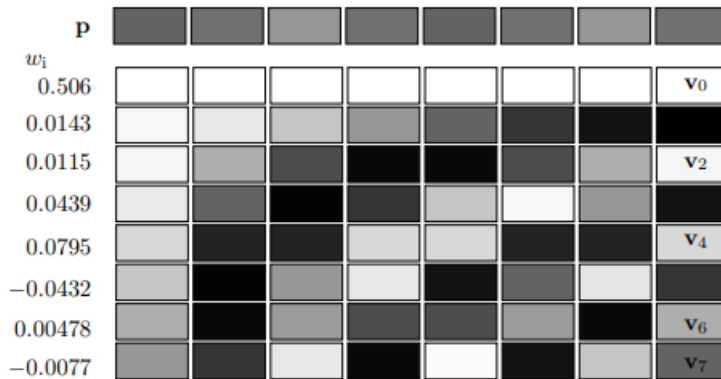


Figure 4: Pixel-based graphical representation of the DCT

For two dimensions, the basis images for an 8x8 input block are shown in Figure 5.

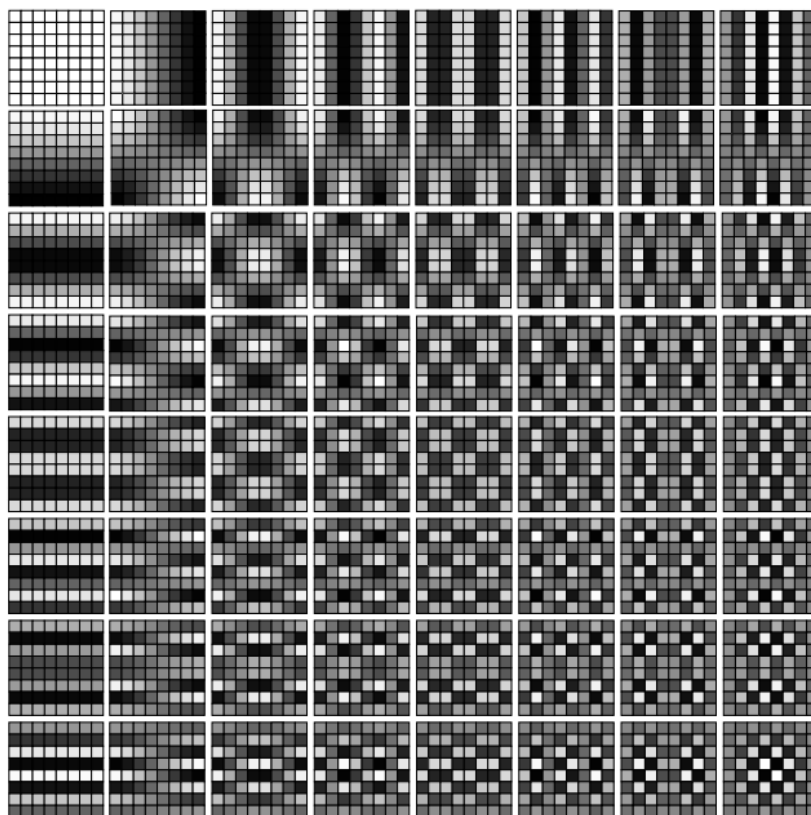


Figure 5: Basis images for the two-dimensional DCT

The two-dimensional DCT for an 8x8 image block is given by:

$$G_{u,v} = -\frac{1}{4}\alpha(u)\alpha(v) \sum_{x=0}^7 \sum_{y=0}^7 p_{x,y} \cos \left[\frac{(2x+1)u\pi}{16} \right] \cos \left[\frac{(2y+1)v\pi}{16} \right]$$

where,

u is the horizontal spatial frequency, for integers $0 \leq u < 8$.

v is the vertical spatial frequency, for integers $0 \leq v < 8$.

$$\alpha(u) = \begin{cases} \frac{1}{\sqrt{2}}, & \text{if } u = 0 \\ 1, & \text{otherwise} \end{cases} \text{ is a normalization scaling factor.}$$

$p_{x,y}$ is the pixel value at coordinates (x, y) .

$G_{u,v}$ is the DCT coefficient at coordinates (u, v) .

Below is an example of a matrix of transformed coefficients from an 8x8 image block. The coefficients indicate that the image is primarily composed of the (0,0) basis image from Figure 5, somewhat less of the (0,1) basis image, and that the small high-frequency coefficients suggest little contribution from the bottom-right basis images.

$$G = \begin{bmatrix} -415.38 & -30.19 & -61.20 & 27.24 & 56.12 & -20.10 & -2.39 & 0.46 \\ 4.47 & -21.86 & -60.76 & 10.25 & 13.15 & -7.09 & -8.54 & 4.88 \\ -46.83 & 7.37 & 77.13 & -24.56 & -28.91 & 9.93 & 5.42 & -5.65 \\ -48.53 & 12.07 & 34.10 & -14.76 & -10.24 & 6.30 & 1.83 & 1.95 \\ 12.12 & -6.55 & -13.20 & -3.95 & -1.87 & 1.75 & -2.79 & 3.14 \\ -7.73 & 2.91 & 2.38 & -5.94 & -2.38 & 0.94 & 4.30 & 1.85 \\ -1.03 & 0.18 & 0.42 & -2.42 & -0.88 & -3.02 & 4.12 & -0.66 \\ -0.17 & 0.14 & -1.07 & -4.19 & -1.17 & -0.10 & 0.50 & 1.68 \end{bmatrix}$$

To conclude this section, it is worth noting that for certain types of images, the DCT reduces pixel values, since in the transformed domain, significant values tend to occur only at lower frequencies. This fulfills the purpose of transforms in the context of image processing as introduced at the beginning of this section: (1) reduce image redundancy by decreasing the size of most pixel values and (2) identify the less important parts of the image by isolating its various frequencies.

3.2 Run Length Encoding

Run Length Encoding (RLE) is a data compression method that is particularly efficient for data containing long sequences of repeated values. The basic principle of RLE is to replace these repeated sequences with a single value and a count indicating how many times the value is repeated.

The RLE algorithm looks for continuous sequences of identical data and, for each repeated sequence, stores the value and the number of repetitions. To reconstruct

the original data, the process is reversed, where each value-count pair is expanded back into the original sequence. For example, if we have a data sequence like:

AAABBBCCDAA

RLE encodes it as:

$3A3B2C1D2A$

Here, $3A$ means that the character A repeats 3 times, $3B$ means that B repeats 3 times, and so on.

Since the 2D DCT returns many small values and many zeros after quantization, it is an ideal target for RLE.

3.3 Huffman Encoding

Huffman Encoding is a lossless data compression method that uses a coding technique based on the frequency of occurrence of symbols in a data set.

The construction of the Huffman tree begins by counting the frequency of each symbol (or character) in the data set. Each symbol is represented by a node in the tree, where the symbol's frequency is the node's weight. The algorithm builds a binary Huffman tree, where each node is either a symbol or an intermediate node with the sum of the frequencies of its children. The nodes with the lowest frequencies are combined first, forming intermediate nodes until all symbols are included in a single tree.

After building the tree, each symbol is encoded with a bit sequence that corresponds to the path from the node to the root of the tree. More frequent symbols are assigned shorter codes, while less frequent symbols receive longer codes.

To decode the data, the process is reversed. The binary code is traversed in the Huffman tree to reconstruct the original symbols. For example, if we have the following symbols and their frequencies:

A: 45
B: 13
C: 12
D: 16
E: 9
F: 5

The Huffman tree may generate codes like:

A: 0
B: 101
C: 100
D: 11
E: 000
F: 001

3.4 JPEG

JPEG is a sophisticated lossy or lossless image compression method for color or grayscale images, which can combine the three topics previously discussed—Huffman Encoding, Run Length Encoding, and the DCT. Other methods may also be used. The main steps of the classic JPEG process are listed below:

- **Data Unit Grouping:** The pixels of each color component are grouped into 8×8 blocks, and each block is compressed separately.
- **Discrete Cosine Transform (DCT):** Each data unit is transformed into a map of frequency components, preparing the data for the lossy stage.
- **Quantization:** Each frequency component is divided by a quantization coefficient and rounded to an integer, which causes irreversible loss of information.
- **Encoding:** The quantized coefficients are encoded using a combination of Run Length Encoding (RLE) and Huffman Encoding, after a zigzag scan of the quantized matrix.

4 JPEG in SoC

4.1 Objective

The goal of this project is to develop a system capable of performing JPEG compression on an 8×8 image block. The image must be transferred to memory to be processed by the DCT module written in VHDL. From the DCT output, the Zynq Processing System in C will handle the subsequent JPEG compression steps. The final result will be evaluated in terms of the accuracy of the algorithms.

4.2 DCT in Hardware

The hardware implementation of the DCT was based on [CSF77]. Figure 6 shows the flow graph of the proposed algorithm for an 8-element sequence.

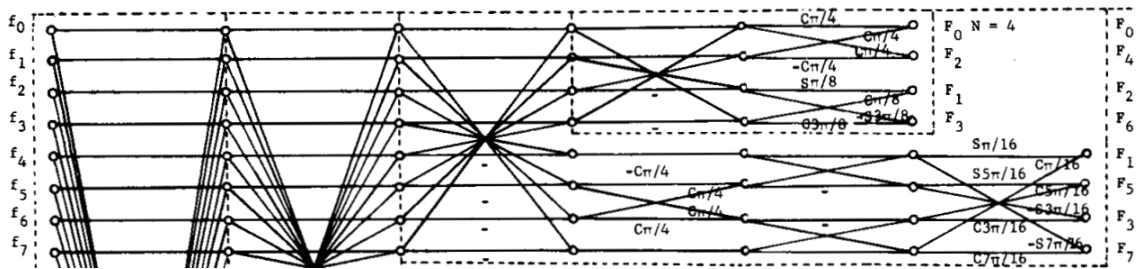


Figure 6: Flow graph of the 8-point DCT algorithm in hardware

To maximize write speed, 8 block memory generators/BRAMs were used to enable parallel read and write operations. Table 1 shows the memory organization, where F_{ij} represents coefficient i of the input sequence j .

Address BRAM	0x00000000	0x00000004	0x00000008	0x0000000C	0x00000010	0x00000014	0x00000018	0x0000001C
BRAM ₀	F_{00}	F_{01}	F_{02}	F_{03}	F_{04}	F_{05}	F_{06}	F_{07}
BRAM ₁	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}	F_{16}	F_{17}
BRAM ₂	F_{20}	F_{21}	F_{22}	F_{23}	F_{24}	F_{25}	F_{26}	F_{27}
BRAM ₃	F_{30}	F_{31}	F_{32}	F_{33}	F_{34}	F_{35}	F_{36}	F_{37}
BRAM ₄	F_{40}	F_{41}	F_{42}	F_{43}	F_{44}	F_{45}	F_{46}	F_{47}
BRAM ₅	F_{50}	F_{51}	F_{52}	F_{53}	F_{54}	F_{55}	F_{56}	F_{57}
BRAM ₆	F_{60}	F_{61}	F_{62}	F_{63}	F_{64}	F_{65}	F_{66}	F_{67}
BRAM ₇	F_{70}	F_{71}	F_{72}	F_{73}	F_{74}	F_{75}	F_{76}	F_{77}

Table 1: BRAM Table

The DCT was implemented in a *pipeline* architecture, so that each layer of the flow graph corresponds to a computation stage, executed every clock cycle.

The implemented hardware DCT receives, at each clock cycle, a sequence of 8 numbers—which, for one-dimensional calculation, would correspond to the rows of the 8x8 image matrix. After computing the DCT for all eight rows, the DCTs of the eight columns are computed, resulting in the final 2D DCT coefficients of the image block.

4.2.1 Timing Considerations

The eight memories were employed to ensure greater parallelism in the system, both for reading and writing data to memory. If a serial read and write approach were used, with a 32-bit data bus, reading and writing a sequence of 8 numbers would take at least 8 clock cycles—the same time the DCT algorithm takes to return the sequence of coefficients for that input. Therefore, a significant amount of time is saved, considering that for image compression, this process would be repeated thousands of times. Compared to the ARM processor, the difference is even greater, as the processor takes 3 clock cycles to perform an addition and up to 4 for a multiplication. These values can be found in tables B1 and B5 of: <https://documentation-service.arm.com/static/5f0370fccafe527e86f5bfb2?token=>

4.3 Block Design

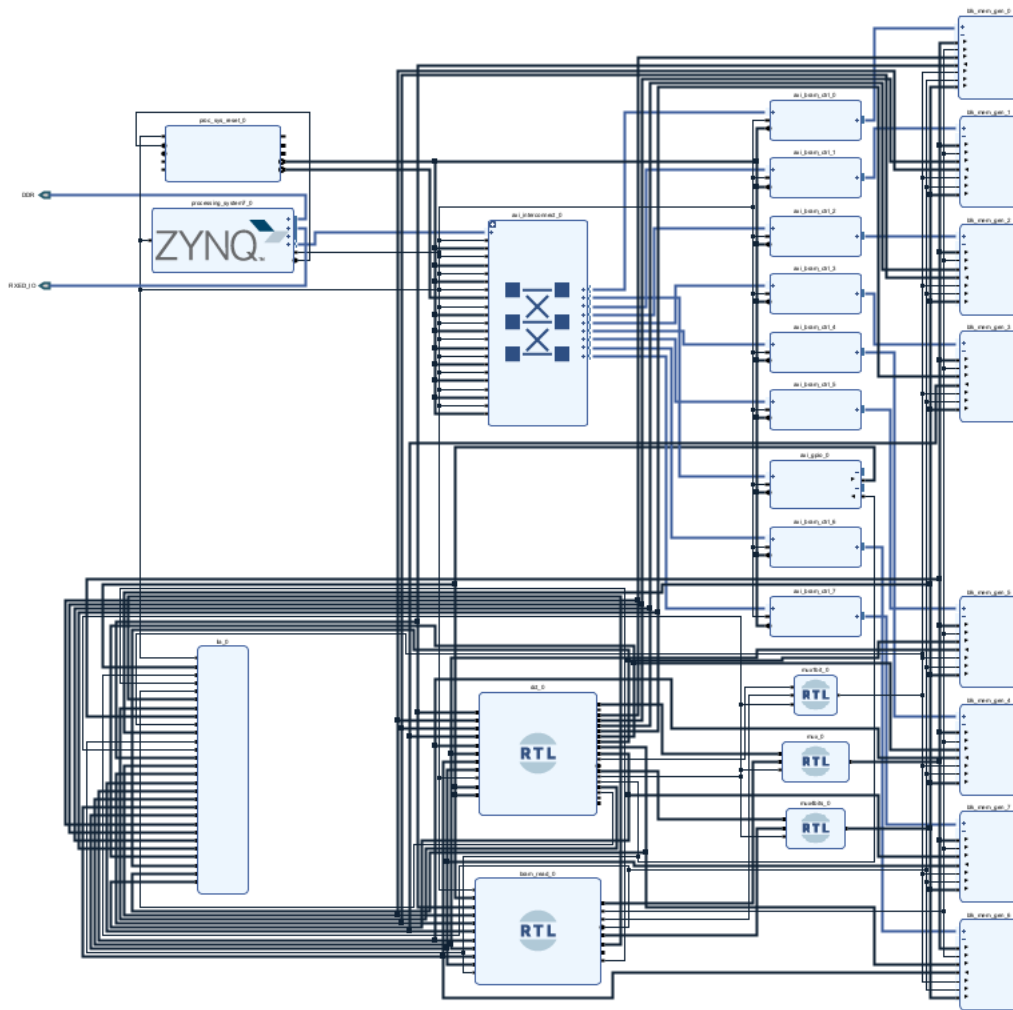


Figure 7: Block Design

4.3.1 DCT

The IP created in Vivado for performing the DCT is shown in Figure 8, where:

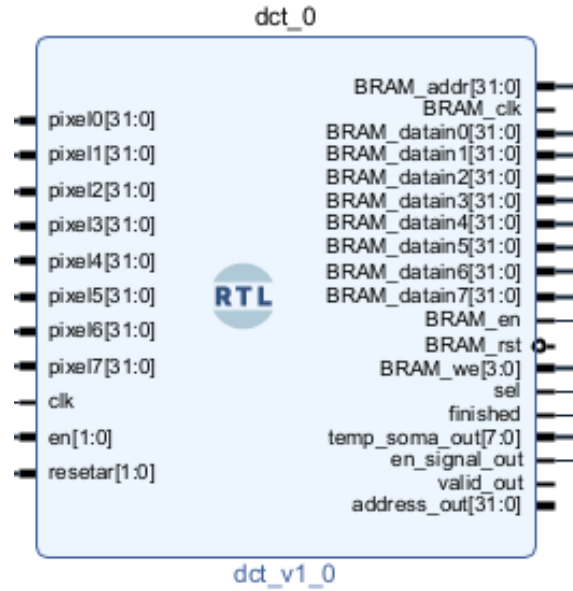


Figure 8: DCT Module

- **pixel0 to pixel7**: Sequence of input pixels.
- **clk**: Clock signal.
- **en[1:0]**: [0] Signal to start the temp_sum count in the transform operation.
- **resetar[1:0]**: [1] Signal to reset the **finished** flag.
- **BRAM_addr[31:0]**: Write address in memory.
- **BRAM_datain0[31:0] to BRAM_datain7[31:0]**: Signals containing the transformed values to be written to BRAM.
- **BRAM_en**: Signal to enable reading or writing (write requires **BRAM_we** as well).
- **BRAM_we[3:0]**: Signal to enable memory write.
- **sel**: Multiplexer selection signal. When **sel=1**, the DCT controls the memory; otherwise, **bram_read** controls it.
- **finished**: Transformation completion signal. Set to 1 when the operation is complete.
- **Other signals**: Debug signals like **BRAM_clk**, **BRAM_rst**, not used.

4.3.2 BRAM Read

The block responsible for reading data from memory is shown in Figure 9, where:

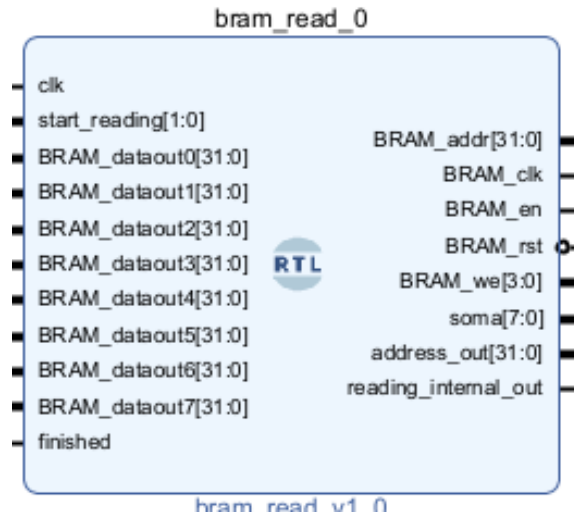


Figure 9: bram_read Module

- **clk**: Clock signal.
- **start_reading[1:0]**: [0] Signal to initiate reading from BRAMs.
- **BRAM_dataout0[31:0] to BRAM_dataout7[31:0]**: Data signals read from the BRAMs.
- **finished**: Transformation completion signal. Equals 1 when complete. Used here to reset internal variables of the block.
- **BRAM_addr[31:0]**: Memory read address.
- **BRAM_clk**: Clock signal for BRAMs, same as **clk**.
- **BRAM_we[3:0]**: Write enable signal. Always set to 0000 here.
- **Other signals**: Debug signals.

4.3.3 BRAM_addr Multiplexer

The multiplexer that selects which component will control the memory addressing is shown in Figure 10, where:



Figure 10: mux Module

- **dct[31:0]**: Address signal from the DCT block.
- **gen[31:0]**: Address signal from the memory read block.

- **y[31:0]**: Selected address signal output.
- **sel**: Selection signal. When 1, selects **dct**; otherwise, selects **gen**.

4.3.4 BRAM_en Multiplexer

The multiplexer for selecting which component controls the BRAM_en signal is shown in Figure 11, where:

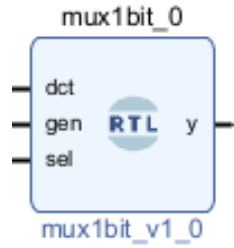


Figure 11: mux1bit Module

- **dct**: BRAM_en signal from the DCT block.
- **gen**: BRAM_en signal from the memory read block.
- **y**: Selected BRAM_en output signal.
- **sel**: Selection signal. When 1, selects **dct**; otherwise, selects **gen**.

4.3.5 BRAM_we Multiplexer

The multiplexer for selecting which component controls the BRAM_we signal is shown in Figure 12, where:

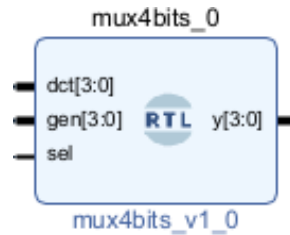


Figure 12: mux4bits Module

- **dct[3:0]**: BRAM_we signal from the DCT block.
- **gen[3:0]**: BRAM_we signal from the memory read block.
- **y[3:0]**: Selected BRAM_we output signal.
- **sel**: Selection signal. When 1, selects **dct**; otherwise, selects **gen**.

4.3.6 Complete Block Design

The complete block design is shown on the last page of this document.

4.4 System Simulation and Operation

4.4.1 Organization

The system is based on the `bram_read`, `dct`, and PS modules. The GPIOs are connected to the PS via the AXI Interconnect, as are the memory blocks. However, the memory blocks contain an additional component to enable communication: the memory controllers, which are connected to the AXI Interconnect. The GPIOs are responsible for reading and writing control signals to the PL that are essential for the project. As explained in Section 4.2, eight memory blocks are used to increase system parallelism. This is not an optimal solution but is better than performing reads and writes serially.

4.4.2 Reading

The simulation of the reading process is shown in Figure 13. Green signals are provided by the GPIO, blue signals belong to `bram_read`, purple signals belong to `dct`, and khaki signals belong to the BRAM.

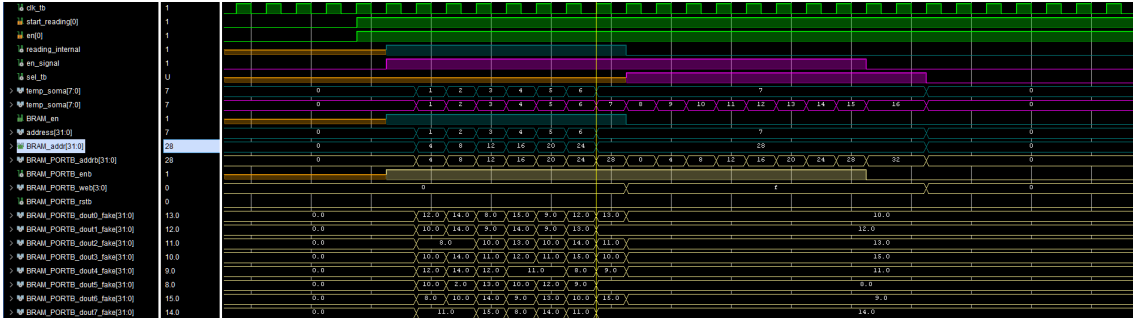


Figure 13: bram_read Simulation

When the `start_reading` signal provided by the GPIO is equal to B"01", the `reading_internal` signal goes high, as does `BRAM_en`, on the rising edge of the clock. The `sel` signal is initially set to zero, which means that `bram_read` controls the memories. When a rising clock edge occurs and `reading_internal` is high, counting begins. The BRAM read address increments progressively by 4. The output data from the BRAM becomes available in the following clock cycle. For example, on the first rising edge with `reading_internal` equal to 1, address 0 is read, and the value is available for use in the next clock cycle. The write signal to BRAM remains low the entire time while `sel` is 0. When `temp_soma` reaches 7, the counting stops and `BRAM_en` goes low. The last data is read at address 28 (decimal) at $t = 145$ ns. The data read are the last eight signals shown in Figure 13.

4.4.3 DCT and Writing

The simulation focusing on the `dct` module is shown below:

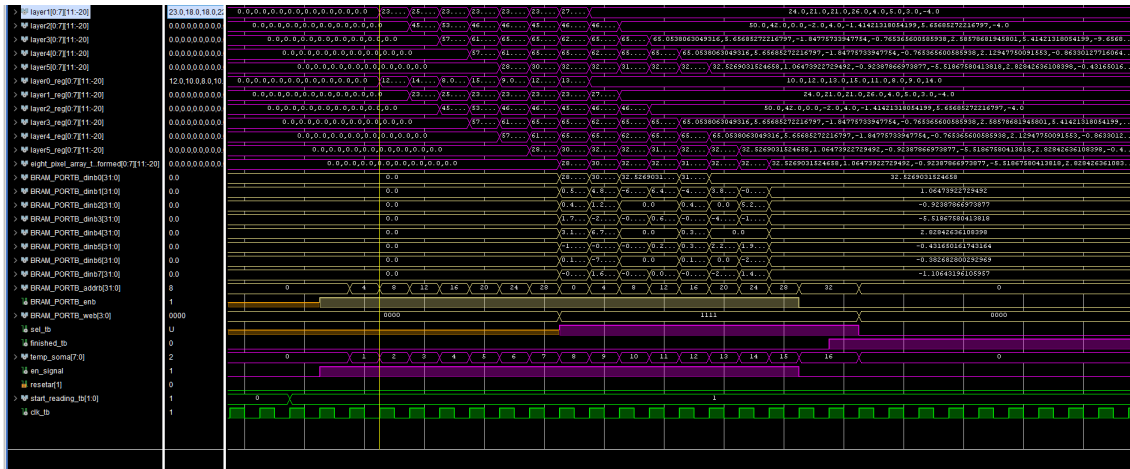


Figure 14: DCT simulation

The data read by `bram_read`—the pixels—are passed to the `layer0_reg` register, which, once filled, is used combinationally by the signal `layer1`, responsible for the first stage of the DCT algorithm. On the next clock cycle, the values of `layer1` are passed to `layer1_reg`, whose values are used by `layer2`, and so on, until `eight_pixel_array_transformed`. The DCT operation is completed once `layer5_reg` is filled; however, an additional register was added to ensure synchronization between the reading process and the DCT. The calculations were done using fixed-point arithmetic: 12 bits for the integer part and sign, and 20 bits for the fractional part.

The `sel` signal is set high once the reading process is completed, passing memory control to the DCT. The memory write latency is one clock cycle. For example, the values shown in Figure 15 are fully written to address 0 only at $t = 165$ ns. That is why the `finished` signal goes high one cycle after `temp_soma` reaches 15. After `finished` is triggered, on the next rising clock edge, all internal control signals are reset. The `finished` signal is only reset when the GPIO sends the `resetar` signal high at $t = 355$ ns.

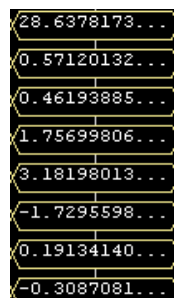


Figure 15: First row DCT coefficients

4.4.4 Software

The software was developed in Vitis and runs on the Zynq Processing System. The code begins by configuring the GPIO and BRAM drivers. Then, each bit direction of the GPIO in each channel is defined. All bits of channel 1 are outputs, and all bits of channel 2 are inputs. Channel 2 receives the `finished` signal. After configuration, the 2D DCT is performed using the `DCT_2D` function on an 8x8 matrix.

The 2D DCT is carried out by first writing each row of the input matrix to memory. Each element of the first row is written to address zero of its respective memory block, the second row to address 4, and so on. These rows are read by the DCT hardware, and the transformed coefficients are written back to memory. At this point, each memory address corresponds to a row in the coefficient matrix. For example, address 0 of memory 0 corresponds to element (0,0) of the coefficient matrix; address 0 of memory 1 corresponds to element (0,1), and so on. This completes the 1D DCT.

Next, each element of the first column of the 1D DCT is written to address zero of the memories, the second column to address 4, and so on. The result is the final 2D DCT coefficient matrix. At this point, address 0 of memory 0 corresponds to element (0,0), address 0 of memory 1 to (1,0), and so forth.

The next step is to divide each element of the DCT-2D coefficient matrix by its corresponding element in the quantization matrix:

$$Q = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

The result of the division is then rounded and followed by zigzag reading.

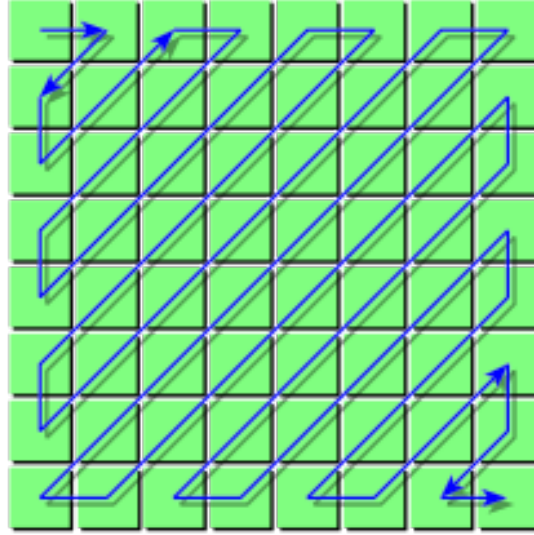


Figure 16: ZigZag

Finally, Run Length Encoding (RLE) and Huffman Encoding are applied. The Huffman implementation was adapted from: <https://www.programiz.com/dsa/huffman-coding>

5 Results

To demonstrate the system's efficiency and accuracy, a simple example was used, also found on Wikipedia [Wik24]. JPEG compression was applied to the following input block:

$$\begin{bmatrix} -76 & -73 & -67 & -62 & -58 & -67 & -64 & -55 \\ -65 & -69 & -73 & -38 & -19 & -43 & -59 & -56 \\ -66 & -69 & -60 & -15 & 16 & -24 & -62 & -55 \\ -65 & -70 & -57 & -6 & 26 & -22 & -58 & -59 \\ -61 & -67 & -60 & -24 & -2 & -40 & -60 & -58 \\ -49 & -63 & -68 & -58 & 51 & -60 & -70 & -53 \\ -43 & -57 & -64 & -69 & 73 & -67 & -63 & -45 \\ -41 & -49 & -59 & -60 & 63 & -52 & -50 & -34 \end{bmatrix}$$

The results match those shown in [Wik24], with only minor differences due to the fixed-point precision used in the developed system.

References

- [CSF77] Wen-Hsiung Chen, C. Smith, and S. Fralick. A fast computational algorithm for the discrete cosine transform. *IEEE Transactions on Communications*, 25(9):1004–1009, 1977.
- [Sal10] David Salomon. *Handbook of Data Compression*. Springer, London; New York, 2010.
- [Wik24] Wikipedia contributors. Jpeg, 2024. Wikipedia, The Free Encyclopedia.